



STUDYSPHERE AI

Intelligent Academic Knowledge Companion

A Project Report on Generative AI, Retrieval-Augmented Generation (RAG),
and Prompt Engineering

Submitted for the
Generative AI Certification Program — ExcelR

Submitted by
Siddharth Singh

Project Type: Full-Stack Generative AI Web Application

Abstract

StudySphere AI is a full-stack Generative AI academic assistant that converts a student's own study materials — lecture notes, textbooks, question papers, and research papers — into an interactive, queryable knowledge base. The system is built around a Retrieval-Augmented Generation (RAG) pipeline: uploaded PDFs are parsed, cleaned, chunked, embedded, and stored directly in a relational database rather than a local file-based index, with similarity search performed via an in-memory FAISS index rebuilt per request. When a student asks a question, the most relevant chunks are retrieved and passed to a Large Language Model along with a prompt shaped by a selected "Study Mode", producing an answer that is grounded in the student's own material and accompanied by page-level source citations. Beyond conversational Q&A, the application offers an AI Exam Assistant, a PDF Summarizer, a Flashcard Generator, an interactive Knowledge Explorer, and a signature AI Study Path feature that produces a day-by-day, personalized revision timeline. The system is implemented with a Python FastAPI backend and a React (Tailwind CSS, Framer Motion) frontend, is fully stateless by design, and is deployed with both the frontend and backend running as Vercel projects from a single repository. It remains fully functional even without a paid LLM subscription through a deterministic extractive fallback mode.

1. Introduction

1.1 Background

The rapid advancement of Large Language Models (LLMs) and Retrieval-Augmented Generation has opened new possibilities for personalized education technology. Students today accumulate large volumes of unstructured PDF material every semester, but generic AI chatbots typically answer from broad internet knowledge rather than the student's own syllabus, which limits their usefulness for exam-focused study and often introduces hallucinated or irrelevant information.

1.2 Problem Statement

- Students possess large amounts of academic PDFs but lack an efficient way to query them directly.
- General-purpose AI assistants are not grounded in a student's specific course material and can hallucinate facts.
- Manually preparing flashcards, revision notes, and structured study schedules from long documents is time-consuming.
- No single tool combines document-grounded Q&A, summarization, exam preparation, and study planning in one workspace.

1.3 Objectives

- Design and implement a Retrieval-Augmented Generation pipeline that grounds AI answers strictly in a student's uploaded documents.
- Provide source citations (document name and page number) for every AI-generated answer to build trust and verifiability.
- Apply Prompt Engineering to let a single AI persona ("ORBIT") adapt its explanation style to different Study Modes.
- Build supporting tools — Exam Assistant, Summarizer, Flashcards, Knowledge Explorer, and AI Study Path — on top of the same RAG core.

- Deliver a secure, multi-user, full-stack web application with authentication and isolated per-user data.

2. Literature Review / Existing Approaches

Most existing "chat with your PDF" tools focus narrowly on single-document question answering and offer little beyond a chat window. General-purpose AI assistants (e.g., broad-domain chatbots) are powerful conversationalists but are not restricted to a user's own source material, which makes them unreliable for exam preparation where factual grounding matters. Retrieval-Augmented Generation, introduced as a way to combine a parametric language model with a non-parametric retrieval mechanism, addresses this by conditioning the model's output on retrieved evidence rather than parametric memory alone. StudySphere AI builds on this idea and extends it beyond simple chat into a complete academic workflow — summarization, exam-question generation, flashcards, topic mapping, and study scheduling — all sourced from the same retrieval layer.

3. Proposed System

3.1 System Overview

StudySphere AI allows a student to sign up, upload one or more PDFs, and interact with that material through six connected tools, all backed by the same per-user vector knowledge base. An AI persona named ORBIT provides a consistent, friendly, and academically-focused voice across every feature.

3.2 Key Features

- **Authentication:** secure sign-up/login with JWT sessions and a private per-user workspace.
- **Smart Document Upload:** drag-and-drop PDF ingestion with an animated processing pipeline (extract → chunk → embed).
- **RAG-Powered AI Chat:** grounded question answering with page-level source citations.
- **Smart Study Modes:** Beginner, Explain Simply, Detailed, Exam Prep, and Quick Revision — each backed by a distinct prompt.
- **AI Exam Assistant:** generates important questions, MCQs, short/long questions, revision notes, and viva questions at three difficulty levels.
- **PDF Summarizer:** short, detailed, or bullet-point summaries of any uploaded document.
- **Flashcard Generator:** AI-generated question/answer flashcards with flip animation and navigation.
- **Knowledge Explorer:** an interactive, animated Subject → Topic → Subtopic map.
- **AI Study Path:** a day-by-day personalized learning and revision timeline, rendered as an animated timeline.
- **Dashboard:** animated statistics for documents uploaded, questions asked, study sessions, and flashcards generated.

4. System Architecture

4.1 High-Level Architecture

The application follows a decoupled client-server architecture. The React frontend communicates with the FastAPI backend exclusively over a REST API secured with JWT bearer tokens. The backend is organized into clearly

separated layers — API routes, business services, the RAG/vector layer, and a dedicated Prompt Management module — which keeps the retrieval-augmented generation flow modular and easy to explain or extend.

4.2 RAG Pipeline

- PDF Upload → written to a temporary file only for the duration of processing, then discarded — nothing is kept on local disk.
- Text Extraction → PyMuPDF extracts text per page from the temporary file.
- Text Cleaning → whitespace normalization and hyphenation repair.
- Chunking → overlapping character-based chunks, each tagged with its source page number.
- Embedding → each chunk is embedded via an OpenAI-compatible embeddings endpoint when an API key is configured, or a deterministic offline hashing-vectorizer otherwise.
- Storage → each chunk and its embedding vector is inserted as a row in the relational database (not a local index file), isolated per user.
- Query Time → the relevant rows are loaded from the database and compared against the query embedding using an in-memory FAISS index built fresh for that request (cosine similarity, top-k).
- Prompt Construction → retrieved chunks are combined with the ORBIT persona prompt and the selected Study Mode instruction.
- Generation → an LLM (or, when no API key is configured, a deterministic extractive fallback) produces the final answer.
- Citation → each answer is returned with the originating document name, page number, and text snippet.

4.3 Data Model

The relational database (SQLite for local development; a managed PostgreSQL instance in production) stores the following core entities. Notably, there is no separate local vector-index file — embeddings are a column in the database itself, which is what allows the backend to run as stateless serverless functions:

- **User:** account credentials (name, email, hashed password) and session data.
- **Document:** uploaded file metadata and processing status (uploading, processing, ready, failed).
- **DocumentChunk:** an embedded chunk of a document — page number, extracted text, and its embedding vector — stored directly as a database row.
- **ChatMessage:** conversation history per user and per document, including retrieved sources and the active study mode.
- **Flashcard:** generated question/answer pairs linked to their source document.
- **StudyPath:** generated day-by-day study plans linked to their source document.

5. Technology Stack

Layer	Technologies Used
Frontend	React 18 (Vite), Tailwind CSS, Framer Motion, Lucide Icons, React Router, Axios
Backend	Python, FastAPI (deployed as Vercel serverless functions), SQLAlchemy ORM, Pydantic

Layer	Technologies Used
Authentication	JSON Web Tokens (python-jose), bcrypt password hashing
Database	SQLite (development) or managed PostgreSQL (production) — a single environment variable switch
PDF Processing	PyMuPDF (fitz) for text extraction, via a temporary file
Embeddings	OpenAI-compatible /embeddings endpoint (text-embedding-3-small) when an API key is configured, with a dependency-free hashing-vectorizer fallback for fully offline use
Vector Search	FAISS (Facebook AI Similarity Search), built in-memory per request from database-stored embeddings
LLM Layer	Any OpenAI-compatible Chat Completions API (OpenAI, Groq, OpenRouter, Gemini-compatible endpoints, etc.)
Prompt Engineering	A dedicated Prompt Management module defining the ORBIT persona and per-mode instructions
Deployment	Both frontend and backend deployed as Vercel projects from the same repository

The embedding strategy was chosen deliberately: a Sentence-Transformers/PyTorch pipeline is capable but adds 700MB-2GB to a deployment, which exceeds typical serverless function size limits. By using an OpenAI-compatible embeddings endpoint (or a lightweight offline hashing-vectorizer fallback) instead, the complete backend installs in well under 500MB, with no first-run model download.

6. Implementation

6.1 Backend Implementation

The backend exposes a modular REST API, with one route module per feature (authentication, documents, chat, summarizer, exam assistant, flashcards, knowledge explorer, study path, and dashboard statistics). Each feature route retrieves the relevant stored chunks for the user from the database and passes them through the shared Prompt Management module before calling the LLM service.

6.2 Stateless, Serverless-Ready Design

A deliberate architectural choice underlies the entire backend: it holds no state on local disk between requests. Uploaded PDFs are written to a temporary file only for the duration of text extraction and are discarded immediately afterward; only the extracted text and its embedding vectors are persisted, as rows in the relational database. This matters because it is what allows the exact same backend code to run identically as a traditional long-running process (for local development) or as short-lived serverless functions on a platform such as Vercel, which provide no persistent filesystem between invocations.

6.3 Frontend Implementation

The frontend is a single-page React application with protected routes, a persistent sidebar, and a shared Documents context so every feature page can operate on the currently selected document. Framer Motion is used throughout for page transitions, an animated AI orb representing ORBIT, upload-pipeline animations, a flashcard flip effect, and an animated study-path timeline.

6.4 Prompt Engineering Module

All system prompts are centralized in a single module rather than scattered inline. A base persona ("ORBIT") instructs the model to answer strictly from the retrieved context and to explicitly state when information is unavailable, reducing hallucination. Five Study Mode instructions — Beginner, Explain Simply, Detailed, Exam Prep, and Quick Revision — are layered on top of this persona depending on the mode selected in the Chat interface, directly demonstrating prompt-engineering technique in a viva-ready, inspectable form.

6.5 Graceful Fallback Without an API Key

To ensure the system is always demonstrable, every generative feature includes a deterministic, dependency-free fallback that activates automatically when no LLM API key is configured. Instead of failing, the application surfaces the most relevant extracted content directly, preserving full end-to-end functionality at zero cost.

7. Testing

7.1 Testing Approach

The backend was validated using an automated, end-to-end functional test covering the full user journey: account creation, PDF upload and processing, RAG-based chat, summarization, exam-question generation, flashcard generation, knowledge-graph generation, study-path generation, and dashboard statistics retrieval. The frontend build was verified with a production Vite build to confirm all components compile and bundle without errors.

7.2 Test Results

Test Case	Expected Result	Status
User sign-up and login	Account created; JWT issued and accepted on subsequent requests	Pass
PDF upload and processing	Document status transitions to "ready" with extracted chunks	Pass
RAG chat query	Answer returned with relevant source citations	Pass
Document summarization	Summary generated in the selected style	Pass
Exam question generation	Questions generated at the selected difficulty and type	Pass
Flashcard generation	Flashcards created and persisted for the document	Pass
Knowledge graph generation	Topic hierarchy returned in the expected structure	Pass
Study path generation	Day-by-day plan returned, including a revision day	Pass
Document deletion	Document, vectors, and related records removed	Pass
Frontend production build	Build completes with no compilation errors	Pass

8. Results and Discussion

All core modules — authentication, document ingestion, RAG-based chat, summarization, exam generation, flashcards, knowledge exploration, and study-path generation — were implemented and verified to function correctly end-to-end. The RAG pipeline successfully restricts answers to retrieved context and returns accurate page-level citations, directly addressing the hallucination concerns identified in the problem statement. The prompt-engineered Study Modes were confirmed to change the tone and structure of ORBIT's responses as intended. The fallback mechanism was also validated, confirming the application remains fully operational without any paid LLM subscription.

9. Deployment

StudySphere AI is engineered so that both the frontend and the backend deploy to the same platform, Vercel, from a single GitHub repository, as two separate Vercel projects sharing one codebase.

9.1 Frontend Deployment — Vercel

The React frontend is a static, build-once single-page application, which is exactly what Vercel is optimized for. The project includes a `vercel.json` configuration specifying the Vite build command, output directory, and a rewrite rule so client-side routing (React Router) continues to work correctly on page refresh and direct links.

9.2 Backend Deployment — Vercel Serverless Functions

Deploying the FastAPI backend to Vercel required removing its dependency on local disk storage — the original design used a local SQLite file and a local FAISS index file, neither of which persist between serverless invocations. The backend was therefore restructured to be fully stateless: every document chunk and its embedding is stored as a row in a relational database rather than a file on disk, and uploaded PDFs are processed through a temporary file that is discarded immediately afterward. With no state left on the local filesystem, the FastAPI application runs unmodified as a Vercel Python serverless function (exposed through a small `api/index.py` entry point and a backend-level `vercel.json`), backed by a managed PostgreSQL database (for example, Vercel's own Postgres integration, or Neon/Supabase) in place of the local SQLite file used during development.

9.3 End-to-End Deployment Flow

- Provision a managed PostgreSQL database (e.g. via Vercel's Storage tab).
- Deploy backend/ as a Vercel project; set `DATABASE_URL` to the Postgres connection string and a secret `APP_SECRET_KEY`.
- Deploy frontend/ as a second Vercel project from the same repository; set `VITE_API_URL` to the deployed backend's URL.
- Set `FRONTEND_ORIGINS` on the backend project to the frontend's URL to restrict CORS to the production frontend only.

9.4 Trade-offs of a Fully Serverless Backend

This design does introduce two practical constraints, both addressed directly in the implementation: serverless functions have a maximum execution duration (configured to 60 seconds here, sufficient for typical assignment-length PDFs), and an in-memory FAISS index must be rebuilt on every request rather than persisting across calls. At the scale of a single student's personal document library — hundreds, not millions, of chunks — rebuilding a flat FAISS index in memory takes single-digit milliseconds, making this a negligible cost relative to the LLM call that

follows it. For deployments that would rather avoid an external managed database altogether, the project also includes a `render.yaml` blueprint, a `Procfile`, and a `Dockerfile` so the identical backend can alternatively be deployed to Render, Railway, or Fly.io with a locally persisted SQLite file, with no code changes required.

10. Conclusion

StudySphere AI demonstrates a complete, practical application of Generative AI, Retrieval-Augmented Generation, and Prompt Engineering to a real academic problem. By grounding every AI response in the student's own uploaded material and citing sources transparently, the system addresses the trust and reliability concerns common to general-purpose AI chatbots, while the surrounding suite of tools — exam preparation, summarization, flashcards, topic mapping, and study planning — turns a simple document chatbot into a complete academic workspace.

11. Future Enhancements

- Adopt a native vector index (e.g. pgvector on PostgreSQL) once document libraries grow large enough that an in-memory FAISS rebuild per request is no longer negligible.
- Support cross-document chat and comparison across multiple uploaded materials simultaneously.
- Stream LLM responses token-by-token over WebSockets for a more responsive chat experience.
- Add OCR support to handle scanned (image-based) PDFs.
- Introduce spaced-repetition scheduling for the Flashcard Generator.
- Enable collaborative, shared study workspaces for group revision.

12. References

Lewis, P. et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.

FastAPI Documentation — <https://fastapi.tiangolo.com>

React Documentation — <https://react.dev>

FAISS: A Library for Efficient Similarity Search — Facebook AI Research.

OpenAI API Documentation (Chat Completions & Embeddings) — <https://platform.openai.com/docs>